# Introduction to Linux Bash programming: 5 `for` loop tips

Bash for loops aren't life-changing for everyone, but they certainly can enhance your productivity.

Posted: March 22, 2021 | 8 min read | Nathan Lager (Sudoer, Red Hat), Ricardo Gerardi (Editorial Team, Sudoer alumni, Red Hat)

*Image by Dimitris Vetsikas from Pixabay*

Every sysadmin probably has some skill they've learned over the years that they can point at and say, "That changed my world." That skill, or that bit of information, or that technique just changed how I do things. For many of us, that thing is *looping* in Bash. There are other approaches to automation that are certainly more robust or scalable. Most of them do not compare to the simplicity and ready usability of the `for` loop, though.

If you want to automate the configuration of thousands of systems, you should probably use Ansible. However, if you're trying to rename a thousand files, or execute the same command several times, then the `for` loop is definitely the right tool for the job.

***[ You might also like: Mastering loops with Jinja templates in Ansible ]***

If you already have a programming or scripting background, you're probably familiar with what `for` loops do. If you're not, I'll try to break it down in plain English for you.

> The basic concept is: **FOR** a given set of items, **DO** a thing.



The given set of items can be a literal set of objects or anything that Bash can extrapolate to a list. For example, text pulled from a file, the output of another Bash command, or parameters passed via the command line. Converting this loop structure into a Bash script is also trivial. In this article, we show you some examples of how a `for` loop can make you look like a command line hero, and then we take some of those examples and put them inside a more structured Bash script.

## Basic structure of the for loop

First, let's talk about the basic structure of a `for` loop, and then we'll get into some examples.

The basic syntax of a `for` loop is:

```
for <variable name> in <a list of items>;do <some command> $<variable name>;done;
```

The **variable name** will be the variable you specify in the `do` section and will contain the item in the loop that you're on.

The **list of items** can be anything that returns a space or newline-separated list.

Here's an example:

```
$ for name in joey suzy bobby;do echo $name;done
```

That's about as simple as it gets and there isn't a whole lot going on there, but it gets you started. The variable **$name** will contain the item in the list that the loop is currently operating on, and once the command (or commands) in the `do` section are carried out, the loop will move to the next item. You can also perform more than one action per loop. Anything between `do` and `done` will be executed. New commands just need a `;` delimiting them.

```
$ for name in joey suzy bobby; do echo first $name;echo second $name;done;
first joey
second joey
first suzy
second suzy
first bobby
second bobby
```

Now for some *real* examples.

# Renaming files

This loop takes the output of the Bash command `ls *.pdf` and performs an action on each returned file name. In this case, we're adding today's date to the end of the file name (but before the file extension).

```
for i in $(ls *.pdf); do
mv $i $(basename $i .pdf)_$(date +%Y%m%d).pdf
done
```

To illustrate, run this loop in a directory containing these files:

```
file1.pdf
file2.pdf
...
fileN.pdf
```

The files will be renamed like this:

```
file1_20210210.pdf
file2_20210210.pdf
...
fileN_20210210.pdf
```

In a directory with hundreds of files, this loop saves you a considerable amount of time in renaming all of them.

# Extrapolating lists of items

Imagine that you have a file that you want to `scp` to several servers. Remember that you can combine the `for` loop with other Bash features, such as shell expansion, which allows Bash to expand a list of items that are in a series. This can work for letters and numbers. For example:

```
$ echo {0..10}
0 1 2 3 4 5 6 7 8 9 10
```

Assuming your servers are named in some sort of pattern like, **web0**, **web1**, **web2**, **web3**, you can have Bash iterate the series of numbers like this:

```
$ for i in web{0..10};do scp somefile.txt ${i}:;done;
```

This will iterate through **web0**, **web1**, **web2**, **web3**, and so forth, executing your command on each item.

You can also define a few iterations. For example:

```
$ for i in web{0..10} db{0..2} balance_{a..c};do echo $i;done
web0
web1
web2
web3
web4
web5
web6
web7
web8
web9
web10
db0
db1
db2
balance_a
balance_b
balance_c
```

You can also combine iterations. Imagine that you have two data centers, one in the United States, another in Canada, and the server's naming convention identifies which data center a server HA pair lived in. For example, **web-us-0** would be the first web server in the US data center, while **web-ca-0** would be **web 0's** counterpart in the CA data center. To execute something on both systems, you can use a sequence like this:

```
$ for i in web-{us,ca}-{0..3};do echo $i;done
web-us-0
web-us-1
web-us-2
web-us-3
web-ca-0
web-ca-1
web-ca-2
web-ca-3
```

In case your server names are not easy to iterate through, you can provide a list of names to the `for` loop:

```
$ cat somelist
first_item
middle_things
foo
bar
baz
last_item

$ for i in `cat somelist`;do echo "ITEM: $i";done
ITEM: first_item
ITEM: middle_things
ITEM: foo
ITEM: bar
ITEM: baz
ITEM: last_item
```

# Nesting

You can also combine some of these ideas for more complex use cases. For example, imagine that you want to copy a list of files to your web servers that follow the numbered naming convention you used in the previous example.

You can accomplish that by iterating a second list based on your first list through nested loops. This gets a little hard to follow when you're doing it as a one-liner, but it can definitely be done. Your nested `for` loop gets executed on every iteration of the parent `for` loop. Be sure to specify different variable names for each loop.

To copy the list of files **file1.txt, file2.txt,** and **file3.txt** to the web servers, use this nested loop:

```
$ for i in file{1..3};do for x in web{0..3};do echo "Copying $i to server $x"; scp $i $x; done; done
Copying file1 to server web0
Copying file1 to server web1
Copying file1 to server web2
Copying file1 to server web3
Copying file2 to server web0
Copying file2 to server web1
Copying file2 to server web2
Copying file2 to server web3
Copying file3 to server web0
Copying file3 to server web1
Copying file3 to server web2
Copying file3 to server web3
```

## More creative renaming

There might be other ways to get this done, but remember, this is just an example of things you *can* do with a `for` loop. What if you have a mountain of files named something like **FILE002.txt**, and you want to replace **FILE** with something like **TEXT**. Remember that in addition to Bash itself, you also have other open source tools at your disposal, like `sed`, `grep`, and more. You can combine those tools with the `for` loop, like this:

```
$ ls FILE*.txt
FILE0.txt  FILE10.txt  FILE1.txt  FILE2.txt  FILE3.txt  FILE4.txt  FILE5.txt  FILE6.txt  FILE7.txt
 FILE8.txt  FILE9.txt

$ for i in $(ls FILE*.txt);do mv $i `echo $i | sed s/FILE/TEXT/`;done

$ ls FILE*.txt
ls: cannot access 'FILE*.txt': No such file or directory

$ ls TEXT*.txt
TEXT0.txt  TEXT10.txt  TEXT1.txt  TEXT2.txt  TEXT3.txt  TEXT4.txt  TEXT5.txt  TEXT6.txt  TEXT7.txt
 TEXT8.txt  TEXT9.txt
```

## Adding a for loop to a Bash script

Running `for` loops directly on the command line is great and saves you a considerable amount of time for some tasks. In addition, you can include `for` loops as part of your Bash scripts for increased power, readability, and flexibility.

For example, you can add the nested loop example to a Bash script to improve its readability, like this:

```
$ vim copy_web_files.sh
# !/bin/bash

for i in file{1..3};do
  for x in web{0..3};do
    echo "Copying $i to server $x"
    scp $i $x
  done
done
```

- Ansible for DevOps
- The ultimate DevOps hiring guide

When you save and execute this script, the result is the same as running the nested loop example above, but it's more readable, plus it's easier to change and maintain.

```
$ bash copy_web_files.sh
Copying file1 to server web0
Copying file1 to server web1
... TRUNCATED ...
Copying file3 to server web3
```

You can also increase the flexibility and reusability of your `for` loops by including them in Bash scripts that allow parameter input. For example, to rename files like the example **More creative renaming** above allowing the user to specify the name suffix, use this script:

```
$ vim rename_files.sh
# !/bin/bash

source_prefix=$1
suffix=$2
destination_prefix=$3

for i in $(ls ${source_prefix}*.${suffix});do
  mv $i $(echo $i | sed s/${source_prefix}/${destination_prefix}/)
done
```

In this script, the user provides the source file's prefix as the first parameter, the file suffix as the second, and the new prefix as the third parameter. For example, to rename all files starting with **FILE**, of type **.txt** to **TEXT**, execute the script like this :

```
$ ls FILE*.txt
FILE0.txt  FILE10.txt  FILE1.txt  FILE2.txt  FILE3.txt  FILE4.txt  FILE5.txt  FILE6.txt  FILE7.txt
 FILE8.txt  FILE9.txt

$ bash rename_files.sh FILE txt TEXT

$ ls TEXT*.txt
TEXT0.txt  TEXT10.txt  TEXT1.txt  TEXT2.txt  TEXT3.txt  TEXT4.txt  TEXT5.txt  TEXT6.txt  TEXT7.txt
 TEXT8.txt  TEXT9.txt
```

This is similar to the original example, but now your users can specify other parameters to change the script behavior. For example, to rename all files now starting with **TEXT** to **NEW**, use the following:

```
$ bash rename_files.sh TEXT txt NEW

$ ls NEW*.txt
NEW0.txt  NEW10.txt  NEW1.txt  NEW2.txt  NEW3.txt  NEW4.txt  NEW5.txt  NEW6.txt  NEW7.txt  NEW8.txt
 NEW9.txt
```



# Conclusion

Hopefully, these examples have demonstrated the power of a `for` loop at the Bash command line. You really can save a lot of time and perform tasks in a less error-prone way with loops. Just be careful. Your loops will do what you ask them to, even if you ask them to do something destructive by accident, like creating (or deleting) logical volumes or

virtual disks.

We hope that Bash `for` loops change your world the same way they changed ours.

## Check out these related articles on Enable Sysadmin


### Bash command line exit codes demystified

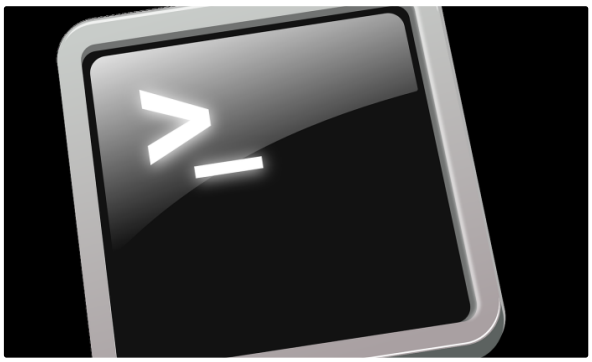
If you've ever wondered what an exit code is or why it's a 0, 1, 2, or even 255, you're in the right place.

Posted: February 4, 2020
Author: Ken Hess (Sudoer alumni)

### Using Bash for automation

Take your Bash scripting to a new higher level using libraries with automation.

Posted: February 13, 2020
Author: Chris Evich (Red Hat)



### Parsing Bash history in Linux

The history command isn't always about reducing key presses. Find out how you can leverage command history into more efficient system administration.

Posted: March 30, 2020
Author: Seth Kenlon (Editorial Team, Red Hat)

**Topics:** Linux Scripting


## Nathan Lager

Nate is a Technical Account Manager with Red Hat and an experienced sysadmin with 20 years in the industry. He first encountered Linux (Red Hat 5.0) as a teenager, after deciding that software licensing was too expensive for a kid with no income, in the late 90's. Since then he's run More about me

## Ricardo Gerardi

Ricardo Gerardi is Technical Community Advocate for Enable Sysadmin and Enable Architect. He was previously a senior consultant at Red Hat Canada, where he specialized in IT automation with Ansible and OpenShift. More about me



## Related Content

### 8 open source 'Easter eggs' to have fun with your Linux terminal

Hunt these 8 hidden or surprising features to make your Linux experience more entertaining.

Posted: April 10, 2023
Author: Ricardo Gerardi (Editorial Team, Sudoer alumni, Red Hat)

### Troubleshooting Linux performance, building a golden image for your RHEL homelab, and more tips for sysadmins

Check out Enable Sysadmin's top 10 articles from March 2023.

Posted: April 4, 2023
Author: Vicki Walker (Editorial Team, Red Hat)

### Do advanced Linux disk usage diagnostics with this sysadmin tool

Use topdiskconsumer to address disk space issues when you're unable to interrupt production.

Posted: March 31, 2023
Author: Kimberly Lazarski (Red Hat)